

haridokei ソースリスト

美都

2026 年 6 月 20 日

目次

1	Rust ソース	2
1.1	メインモジュール main.rs	2
1.2	初期化 init.rs	4
1.3	時計ウィンドウの表示 clock_window.rs	8
1.4	時計エレメント clock_elm.rs	10
1.5	時計のベースイメージの生成 clock_base_image.rs	14
1.6	アプリケーションの設定情報 global_setting.rs	19
1.7	アプリケーションの引数管理 options.rs	21

1 Rust ソース

1.1 メインモジュール main.rs

```
1  mod clock_base_image;
2  mod clock_elm;
3  mod clock_window;
4  mod global_setting;
5  mod init;
6  mod options;
7
8  use std::sync::{Arc, Mutex};
9
10 use anyhow::Result;
11 use clap::Parser;
12 use log::{error, info};
13
14 use crate::{
15     clock_window::open_main_window,
16     global_setting::{GlobalSetting, GlobalSettingFile},
17     init::notify_systemd_ready,
18 };
19
20 fn main() -> Result<> {
21     let options = options::Options::parse(); // コマンドライン引数の処理
22     init::init_logging(&options); // ロギング機構初期化
23
24     // 設定ファイルの読み込み
25     let global_setting_file_read: GlobalSettingFile = confy::load(global_setting::APP_NAME, None)
26         .unwrap_or_else(|e| {
27             error!("設定ファイルの読み込みに失敗。デフォルト値を使用します。:{e}");
28             GlobalSettingFile::default()
29         });
30     let global_setting_file = Arc::new(Mutex::new(global_setting_file_read));
31
32     let mut pid_file = init::PidFile::new().inspect_err(|e| {
33         error!("PID ファイルの生成に失敗しました。:{e:?}");
34     })?;
35     if options.daemon {
36         init::do_daemonize(&mut pid_file).inspect_err(|e| {
37             error!("デーモン化に失敗しました。:{e:?}");
38         })?;
39     };
40
41     let is_terminate = init::register_signal_handler();
```

```

42     let global_setting = GlobalSetting::new(global_setting_file.clone(), is_terminate);
43
44     info!("針時計を開始しました。");
45
46     // アプリケーションの実行
47     gpui_platform::application().run(move |app| {
48         app.set_global(global_setting);
49         open_main_window(app);
50
51         notify_systemd_ready();
52     });
53
54     // 設定ファイルの保存。
55     match confy::store(
56         global_setting::APP_NAME,
57         None,
58         global_setting_file.lock().unwrap().clone(),
59     ) {
60         Ok(_) => info!("設定ファイルを保存しました。"),
61         Err(e) => error!("設定ファイルの保存に失敗しました。:{e}"),
62     };
63     info!("針時計を終了しました。");
64     Ok(())
65 }

```

1.2 初期化 init.rs

```
1  /// アプリケーションの初期化関連手続き
2
3  use crate::options::Options;
4
5  use anyhow::{Context, Result};
6  use log::{error, info, warn};
7  use std::{
8      fs::{File, OpenOptions},
9      io::{Seek, Write},
10     path::PathBuf,
11     sync::{Arc, atomic::AtomicBool},
12 };
13 use systemd_journal_logger::JournalLog;
14
15 /// ロギング機構の初期化を行う。
16 pub fn init_logging(options: &Options) {
17     JournalLog::new()
18         .unwrap()
19         .with_syslog_identifier("haridokei".to_string())
20         .install()
21         .unwrap();
22     if options.debug {
23         log::set_max_level(log::LevelFilter::Debug);
24     } else if options.info {
25         log::set_max_level(log::LevelFilter::Info);
26     } else {
27         log::set_max_level(log::LevelFilter::Warn);
28     }
29 }
30
31 /// 指定されていればデーモン化する
32 pub fn do_daemonize(pid_file: &mut PidFile) -> Result<()> {
33     let daemonize = daemonize::Daemonize::new().chown_pid_file(true);
34     daemonize.start().context("デーモン化に失敗しました。")?;
35     info!("デーモン化に成功しました。");
36     pid_file
37         .update_pid()
38         .context("PID ファイルの更新に失敗しました。")?;
39     Ok(())
40 }
41
42 /// シグナルハンドラの登録を行う。
43 /// 終了シグナル受信用のフラグを返す。
44 pub fn register_signal_handler() -> Arc<AtomicBool> {
```

```

45     let is_terminate = Arc::new(AtomicBool::new(false));
46     signal_hook::flag::register(signal_hook::consts::SIGINT, is_terminate.clone())
47         .map(|_id| ())
48         .unwrap_or_else(|e| error!("シグナルハンドラの登録に失敗しました。(SIGINT):{}", e));
49     signal_hook::flag::register(signal_hook::consts::SIGTERM, is_terminate.clone())
50         .map(|_id| ())
51         .unwrap_or_else(|e| error!("シグナルハンドラの登録に失敗しました。(SIGTERM):{}", e));
52     is_terminate
53 }
54
55 /// systemdに起動完了を通知する。
56 pub fn notify_systemd_ready() {
57     let window_initialized_message = "ウィンドウの初期化を終了しました。";
58     info!("{}", window_initialized_message);
59     sd_notify::notify(&[
60         sd_notify::NotifyState::Ready,
61         sd_notify::NotifyState::Status(window_initialized_message),
62     ])
63     .unwrap_or_else(|e| {
64         error!("systemd への通知に失敗しました。:{}", e);
65     });
66 }
67
68 /// systemdに終了を通知する。
69 pub fn notify_systemd_stopping() {
70     let stopping_message = "針時計を終了します。";
71     sd_notify::notify(&[
72         sd_notify::NotifyState::Stopping,
73         sd_notify::NotifyState::Status(stopping_message),
74     ])
75     .unwrap_or_else(|e| {
76         error!("systemd への通知に失敗しました。:{}", e);
77     });
78     info!("{}", stopping_message);
79 }
80
81 /// PID ファイルの維持管理を行う
82 pub struct PidFile {
83     file: File,
84     file_path: PathBuf,
85     is_enable: bool,
86 }
87
88 impl PidFile {
89     const PID_FILE_NAME: &str = "haridokei.pid";
90

```

```

91 pub fn new() -> Result<Self> {
92     let path = Self::get_pid_file_path().context("PID ファイルのパスの取得に失敗しました。")?;
93     let file = OpenOptions::new()
94         .write(true)
95         .create(true)
96         .truncate(false) // ロック取得の確定後、明示的に、set_len(0) で消すため、ここでは、false と
           ↳ する。
97         .open(&path)
98         .context("PID ファイルの生成に失敗しました。")?;
99     let mut ret = Self {
100         file,
101         file_path: path.to_path_buf(),
102         is_enable: false,
103     };
104     ret.update_pid()
105         .context("PID ファイルの PID 登録に失敗しました。")?;
106     ret.is_enable = true;
107     Ok(ret)
108 }
109
110 /// PID ファイルのパスを取得する。
111 /// ファイルの生成場所は `XDG_RUNTIME_DIR`。環境変数がない場合、`/tmp`。
112 /// もし、既存のファイルが存在する場合、ロックの可否を確認し、ロックが取れないからエラーを返す。
113 fn get_pid_file_path() -> Result<PathBuf> {
114     let pid_path = match std::env::var("XDG_RUNTIME_DIR") {
115         Ok(runtime_dir) if !runtime_dir.trim().is_empty() => {
116             let mut path = PathBuf::from(runtime_dir);
117             path.push(Self::PID_FILE_NAME);
118             path
119         }
120         Ok(_) | Err(_) => PathBuf::from(format!("/tmp/{}", Self::PID_FILE_NAME)),
121     };
122     Ok(pid_path)
123 }
124
125 /// PID ファイルに登録の PID を現在のものに更新する。
126 pub fn update_pid(&mut self) -> Result<()> {
127     self.file
128         .try_lock()
129         .context("PID ファイルのロックに失敗しました。二重起動の可能性があります。")?;
130     self.file.set_len(0)?;
131     self.file.rewind()?;
132     write!(self.file, "{}", std::process::id())?;
133     self.file.flush()?;
134     self.file.try_lock_shared()?;
135     Ok(())

```

```
136     }
137 }
138
139 impl Drop for PidFile {
140     fn drop(&mut self) {
141         if self.is_enable {
142             if let Err(e) = std::fs::remove_file(&self.file_path) {
143                 error!("PID ファイルの削除に失敗しました。:{}", e);
144             }
145             info!("PID ファイルを削除しました。");
146         } else {
147             warn!("PID ファイルは有効化されていないため、削除は行いませんでした。");
148         };
149     }
150 }
```

1.3 時計ウィンドウの表示 clock_window.rs

```
1 use std::sync::{Arc, Mutex, atomic::AtomicBool};
2
3 use gpui::{
4     App, AsyncApp, Pixels, Size, TitlebarOptions, WindowBounds, WindowOptions, prelude::*, px,
5     ⇨ size,
6 };
7
8 use log::info;
9
10 use crate::{clock_base_image::ClockBaseImage, clock_elm::Clock, global_setting::GlobalSetting};
11
12 /// タイトルバーの表示名
13 const TITLE_NAME: &str = "Haridokei";
14
15 /// アプリケーション ID(WM_CLASSに使用される)
16 const APP_ID: &str = "jp.laki.haridokei";
17
18 /// メインウィンドウをオープンする
19 pub fn open_main_window(app: &mut App) {
20     let global_setting: &GlobalSetting = app.global();
21     // メインウィンドウの生成
22     let titlebar = TitlebarOptions {
23         title: Some(TITLE_NAME.into()),
24         ..Default::default()
25     };
26
27     let win_opt = WindowOptions {
28         window_bounds: Some(WindowBounds::Windowed(global_setting.bounds())),
29         window_min_size: Some(size(px(50.), px(50.))),
30         titlebar: Some(titlebar),
31         app_id: Some(APP_ID.into()),
32         ..Default::default()
33     };
34
35     app.open_window(win_opt, |win, cx| {
36         cx.new(|cx| {
37             let win_size = win.bounds().size;
38             let global_setting: &GlobalSetting = cx.global();
39             let clock_background_color = global_setting.clock_background_color();
40             ClockWindow::new(
41                 win_size,
42                 clock_background_color,
43                 global_setting.is_terminated(),
44             )
45         })
46     })
47     .unwrap();
48 }
```



```

44
45 /// 時計を表示するコンテキスト
46 pub struct ClockWindow {
47     base_image: Arc<Mutex<ClockBaseImage>>,
48     is_terminate: Arc<AtomicBool>,
49 }
50
51 impl ClockWindow {
52     pub fn new(
53         size: Size<Pixels>,
54         background_color: [u8; 4],
55         is_terminate: Arc<AtomicBool>,
56     ) -> Self {
57         Self {
58             base_image: Arc::new(Mutex::new(ClockBaseImage::new(size, background_color))),
59             is_terminate,
60         }
61     }
62 }
63
64 impl Render for ClockWindow {
65     fn render(&mut self, window: &mut gpui::Window, cx: &mut Context<Self>) -> impl IntoElement {
66         if self.is_terminate.load(std::sync::atomic::Ordering::Relaxed) {
67             crate::init::notify_systemd_stopping();
68             info!("終了シグナルを受信しました。終了処理に入ります。");
69             cx.quit();
70         }
71         let win_size = window.bounds().size;
72         let entity_id = cx.entity_id();
73         cx.spawn(async move |_, cx: &mut AsyncApp| {
74             cx.background_executor()
75                 .timer(std::time::Duration::from_millis(33))
76                 .await;
77             cx.update(|cx| {
78                 cx.notify(entity_id);
79             });
80         })
81         .detach();
82         self.base_image.lock().unwrap().set_size(win_size);
83         cx.global_mut::<GlobalSetting>().set_bounds(window.bounds());
84         Clock::new(self.base_image.clone())
85     }
86 }

```

1.4 時計エレメント clock_elm.rs

```
1 use std::sync::{Arc, Mutex};
2
3 use chrono::{Local, Timelike};
4 use gpui::{App, Bounds, PathBuilder, Pixels, Point, Style, Window, fill, prelude::*, px, rgba};
5
6 use crate::clock_base_image::ClockBaseImage;
7
8 /// 時計を表示するための Element
9 pub struct Clock {
10     base_image: Arc<Mutex<ClockBaseImage>>,
11     center: Point<Pixels>,
12     radius: Pixels,
13 }
14
15 impl Clock {
16     pub fn new(base_image: Arc<Mutex<ClockBaseImage>>) -> Self {
17         let (center, radius) = {
18             let base_image = base_image.lock().unwrap();
19             (base_image.center(), px(base_image.radius()))
20         };
21         Self {
22             base_image,
23             center,
24             radius,
25         }
26     }
27
28     /// 秒針の描画
29     fn paint_second_hand(
30         &mut self,
31         bounds: Bounds<Pixels>,
32         window: &mut Window,
33         second: f32,
34         millis: f32,
35     ) {
36         let arg = (360. / 60.) * second + (360. / (60. * 1000.)) * millis;
37         let mut path = PathBuilder::fill();
38         path.move_to(bounds.origin);
39         path.line_to(Point::new(-(self.radius * 0.01), -(self.radius * 0.3)));
40         path.line_to(Point::new(px(0.0), -(self.radius * 0.9)));
41         path.line_to(Point::new(self.radius * 0.01, -(self.radius * 0.3)));
42         path.close();
43         path.rotate(arg);
44         path.translate(self.center);
```

```

45     let path = path.build().unwrap();
46     window.paint_path(path, rgba(0x000000ff));
47 }
48
49 /// 長針の描画
50 fn paint_long_hand(
51     &mut self,
52     bounds: Bounds<Pixels>,
53     window: &mut Window,
54     minute: f32,
55     second: f32,
56     millis: f32,
57 ) {
58     let arg = (360. / 60.) * minute
59         + ((360. / (60. * 60.)) * second + (360. / (60. * 60. * 1000.)) * millis);
60     let mut path = PathBuilder::fill();
61     path.move_to(bounds.origin);
62     path.line_to(Point::new(-(self.radius * 0.04), -(self.radius * 0.3)));
63     path.line_to(Point::new(px(0.0), -(self.radius * 0.9)));
64     path.line_to(Point::new(self.radius * 0.04, -(self.radius * 0.3)));
65     path.close();
66     path.rotate(arg);
67     path.translate(self.center);
68     let path = path.build().unwrap();
69     window.paint_path(path, rgba(0x000000ff));
70 }
71
72 /// 短針の描画
73 fn paint_short_hand(
74     &mut self,
75     bounds: Bounds<Pixels>,
76     window: &mut Window,
77     hour: f32,
78     minute: f32,
79     second: f32,
80 ) {
81     let arg = (360. / 12.) * hour
82         + ((360. / (12. * 60.)) * minute + (360. / (12. * 60. * 60.)) * second);
83     let mut path = PathBuilder::fill();
84     path.move_to(bounds.origin);
85     path.line_to(Point::new(-(self.radius * 0.06), -(self.radius * 0.2)));
86     path.line_to(Point::new(px(0.0), -(self.radius * 0.55)));
87     path.line_to(Point::new(self.radius * 0.06, -(self.radius * 0.2)));
88     path.close();
89     path.rotate(arg);
90     path.translate(self.center);

```

```

91     let path = path.build().unwrap();
92     window.paint_path(path, rgba(0x000000ff));
93 }
94 }
95
96 impl Element for Clock {
97     type RequestLayoutState = ();
98     type PrepaintState = ();
99
100     fn paint(
101         &mut self,
102         _id: Option<&gpui::GlobalElementId>,
103         _inspector_id: Option<&gpui::InspectorElementId>,
104         bounds: Bounds<Pixels>,
105         _request_layout: &mut Self::RequestLayoutState,
106         _prepaint: &mut Self::PrepaintState,
107         window: &mut Window,
108         cx: &mut App,
109     ) {
110         let now = Local::now();
111         let (_, hour) = now.hour12();
112         let hour = (hour % 12) as f32;
113         let minute = now.minute() as f32;
114         let second = now.second() as f32;
115         let millis = now.timestamp_subsec_millis() as f32;
116
117         // windowのクリア
118         let size = bounds.size;
119         let bounds = Bounds {
120             origin: Point::new(px(0.0), px(0.0)),
121             size,
122         };
123         let rect = fill(bounds, rgba(0x00000000));
124         window.paint_quad(rect);
125         // 時計のベースイメージを描画
126         self.base_image.lock().unwrap().paint_clock_base(window, cx);
127         // 針の描画
128         self.paint_second_hand(bounds, window, second, millis);
129         self.paint_long_hand(bounds, window, minute, second, millis);
130         self.paint_short_hand(bounds, window, hour, minute, second);
131     }
132
133     fn source_location(&self) -> Option<&'static std::panic::Location<'static>> {
134         None
135     }
136

```

```

137     fn id(&self) -> Option<gpui::ElementId> {
138         None
139     }
140
141     fn request_layout(
142         &mut self,
143         _id: Option<&gpui::GlobalElementId>,
144         _inspector_id: Option<&gpui::InspectorElementId>,
145         window: &mut Window,
146         cx: &mut App,
147     ) -> (gpui::LayoutId, Self::RequestLayoutState) {
148         let style = Style {
149             size: gpui::Size::full(),
150             ..Default::default()
151         };
152         let layoutid = window.request_layout(style, None, cx);
153         (layoutid, ())
154     }
155
156     fn prepaint(
157         &mut self,
158         _id: Option<&gpui::GlobalElementId>,
159         _inspector_id: Option<&gpui::InspectorElementId>,
160         _bounds: gpui::Bounds<gpui::Pixels>,
161         _request_layout: &mut Self::RequestLayoutState,
162         _window: &mut Window,
163         _cx: &mut App,
164     ) -> Self::PrepaintState {
165     }
166 }
167
168 impl IntoElement for Clock {
169     type Element = Self;
170     fn into_element(self) -> Self::Element {
171         self
172     }
173 }

```

1.5 時計のベースイメージの生成 clock_base_image.rs

```
1 use std::f32::consts::PI;
2
3 use gpui::{
4     App, Font, FontFallbacks, FontFeatures, FontStyle, FontWeight, Path, PathBuilder, Pixels,
5     Point, Rgba, Size, TextAlign, TextRun, Window, px, rgba,
6 };
7 use log::error;
8
9 pub struct ClockBaseImage {
10     size: Size<Pixels>,
11     center: Point<Pixels>,
12     radius: f32,
13     clock_background_color: Rgba,
14     img_path: Vec<(Path<Pixels>, Rgba)>,
15 }
16
17 impl ClockBaseImage {
18     pub fn new(size: Size<Pixels>, clock_background_color: [u8; 4]) -> Self {
19         let center = Point::new(size.width / 2.0, size.height / 2.0);
20         let radius = size.width.min(size.height).as_f32() / 2.0;
21         let color_value = clock_background_color
22             .iter()
23             .fold(0, |acc, &c| (acc << 8) + c as u32);
24         let back_ground_color = gpui::rgba(color_value);
25         let img_path = Vec::new();
26
27         let mut obj = Self {
28             size,
29             center,
30             radius,
31             img_path,
32             clock_background_color: back_ground_color,
33         };
34         obj.make_clock_base();
35         obj
36     }
37
38     /// 時計背景のイメージを描画する。
39     pub fn paint_clock_base(&self, window: &mut gpui::Window, cx: &mut gpui::App) {
40         for (path, color) in self.img_path.iter() {
41             window.paint_path(path.clone(), *color);
42         }
43         self.draw_numbers(window, cx);
44     }
45 }
```

```

45
46 pub fn center(&self) -> Point<Pixels> {
47     self.center
48 }
49
50 pub fn radius(&self) -> f32 {
51     self.radius
52 }
53
54 pub fn set_size(&mut self, size: Size<Pixels>) {
55     if self.size != size {
56         self.size = size;
57         self.center = Point::new(size.width / 2.0, size.height / 2.0);
58         self.radius = size.width.min(size.height).as_f32() / 2.0;
59         self.make_clock_base();
60     }
61 }
62
63 /// 時計の背景のイメージを生成する。
64 fn make_clock_base(&mut self) {
65     self.img_path.clear();
66     self.draw_clock_background();
67     self.draw_major_scale();
68     self.draw_center_pin();
69     self.draw_miner_scale();
70 }
71
72 fn draw_clock_background(&mut self) {
73     let mut path_bld = PathBuilder::fill();
74     self.draw_circle(&mut path_bld, self.center, self.radius);
75     match path_bld.build() {
76         Ok(path) => self.img_path.push((path, self.clock_background_color)),
77         Err(e) => error!("時計背景の丸の描画に失敗 ({e})"),
78     }
79 }
80
81 /// 大目盛りの描画を行う
82 fn draw_major_scale(&mut self) {
83     let width = Point::new(px(self.radius / 10.0), px(0.0));
84     let height = Point::new(px(0.0), px(self.radius / 30.0));
85     let first_point = Point::new(px(self.radius * 0.98), height.y / 2.0);
86     for i in 0..12 {
87         let mut path_bld = PathBuilder::fill();
88         path_bld.move_to(first_point);
89         path_bld.line_to(first_point - height);
90         path_bld.line_to(first_point - height - width);

```

```

91     path_bld.line_to(first_point - width);
92     path_bld.close();
93     let angle = (360.0 / 12.0) * i as f32;
94     path_bld.rotate(angle);
95     path_bld.translate(self.center);
96     match path_bld.build() {
97         Ok(path) => self.img_path.push((path, gpui::rgba(0x000000ff))),
98         Err(e) => error!("大目盛りの描画に失敗 ({e})"),
99     }
100 }
101 }
102
103 /// センターピンの描画を行う
104 fn draw_center_pin(&mut self) {
105     let mut path_bld = PathBuilder::fill();
106     self.draw_circle(&mut path_bld, self.center, self.radius / 20.0);
107     match path_bld.build() {
108         Ok(path) => self.img_path.push((path, gpui::rgba(0x000000ff))),
109         Err(e) => error!("センターピンの描画に失敗 ({e})"),
110     }
111 }
112
113 /// 小目盛りの描画を行う
114 fn draw_miner_scale(&mut self) {
115     let first_point = Point::new(px(self.radius * 0.93), px(0.0));
116     let scale_radius = self.radius / 60.0;
117     let indexs = (0..60).filter(|i| i % 5 != 0);
118     for i in indexs {
119         let mut path_bld = PathBuilder::fill();
120         let angle = (360.0 / 60.0) * i as f32;
121         self.draw_circle(&mut path_bld, first_point, scale_radius);
122         path_bld.rotate(angle);
123         path_bld.translate(self.center);
124         match path_bld.build() {
125             Ok(path) => self.img_path.push((path, gpui::rgba(0x000000ff))),
126             Err(e) => error!("小目盛りの描画に失敗 ({e})"),
127         }
128     }
129 }
130
131 /// 文字盤の数字を描画する
132 fn draw_numbers(&self, window: &mut Window, cx: &mut App) {
133     const FONT_SIZE_RATE: f32 = 0.25;
134     const NUM_POSITION_RATE: f32 = 0.70;
135     let font_size = px(self.radius() * FONT_SIZE_RATE);
136     let font = Font {

```



```

137         family: "Fraunces".into(),
138         features: FontFeatures::default(),
139         fallbacks: Some(FontFallbacks::from_fonts(vec![".SystemUIFont".into()])),
140         weight: FontWeight::default(),
141         style: FontStyle::default(),
142     };
143     let num_strs = [
144         "12", "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11",
145     ];
146     let mut text_run = TextRun {
147         len: 0,
148         font,
149         color: rgba(0x000000ff).into(),
150         background_color: None,
151         underline: None,
152         strikethrough: None,
153     };
154     for (i, num_str) in num_strs.iter().enumerate() {
155         let angle = 2.0 * PI * i as f32 / 12.0;
156         text_run.len = num_str.len();
157         let shaped_line = window.text_system().shape_line(
158             (*num_str).into(),
159             font_size,
160             std::slice::from_ref(&text_run),
161             None,
162         );
163         let x = (self.radius() * NUM_POSITION_RATE) * f32::sin(angle);
164         let y = -(self.radius() * NUM_POSITION_RATE) * f32::cos(angle);
165         let mut origin = Point::new(px(x), px(y)) + self.center;
166         origin += Point::new(-shaped_line.width() / 2.0, -font_size / 2.0);
167         shaped_line
168             .paint(
169                 origin,
170                 font_size,
171                 TextAlign::default(),
172                 Some(shaped_line.width()),
173                 window,
174                 cx,
175             )
176             .unwrap_or_else(|e| error!("文字盤の数字の描画に失敗しました。:({e})"));
177     }
178 }
179
180 /// 円を描画する
181 fn draw_circle(&mut self, path_bld: &mut PathBuilder, center: Point<Pixels>, radius: f32) {
182     let point_up = Point::new(px(0.0), px(radius));

```

```
183     let point_down = Point::new(px(0.0), px(-radius));
184     let radii = Point::new(px(radius), px(radius));
185     path_bld.move_to(point_up);
186     path_bld.arc_to(radii, px(0.0), true, true, point_down);
187     path_bld.arc_to(radii, px(0.0), true, true, point_up);
188     path_bld.translate(center);
189 }
190 }
```

1.6 アプリケーションの設定情報 global_setting.rs

```
1 use std::sync::{Arc, Mutex, atomic::AtomicBool};
2
3 use gpui::{Bounds, Pixels, point, px, size};
4 use serde::{Deserialize, Serialize};
5
6 pub const APP_NAME: &str = env!("CARGO_PKG_NAME");
7
8 #[derive(Debug, Clone)]
9 pub struct GlobalSetting {
10     save_item: Arc<Mutex<GlobalSettingFile>>,
11     is_terminated: Arc<AtomicBool>,
12 }
13
14 impl GlobalSetting {
15     pub fn new(in_file: Arc<Mutex<GlobalSettingFile>>, is_terminated: Arc<AtomicBool>) -> Self {
16         Self {
17             save_item: in_file,
18             is_terminated: is_terminated.clone(),
19         }
20     }
21
22     pub fn bounds(&self) -> Bounds<Pixels> {
23         self.save_item.lock().unwrap().bounds
24     }
25
26     pub fn clock_background_color(&self) -> [u8; 4] {
27         self.save_item.lock().unwrap().clock_background_color
28     }
29
30     pub fn set_bounds(&mut self, bounds: Bounds<Pixels>) {
31         self.save_item.lock().unwrap().bounds = bounds;
32     }
33
34     pub fn is_terminated(&self) -> Arc<AtomicBool> {
35         self.is_terminated.clone()
36     }
37 }
38
39 impl gpui::Global for GlobalSetting {}
40
41 /// グローバルセッティングのうち、ファイルに保存するための値
42 #[derive(Serialize, Deserialize, Debug, Clone)]
43 #[serde(default)]
44 pub struct GlobalSettingFile {
```

```

45     bounds: Bounds<Pixels>,
46     clock_background_color: [u8; 4],
47 }
48
49 impl Default for GlobalSettingFile {
50     fn default() -> Self {
51         let size = size(px(300.), px(300.));
52         let origin = point(px(10.), px(10.));
53         Self {
54             bounds: Bounds::new(origin, size),
55             clock_background_color: [0, 255, 255, 255],
56         }
57     }
58 }
59
60 impl From<GlobalSetting> for GlobalSettingFile {
61     fn from(value: GlobalSetting) -> Self {
62         value.save_item.lock().unwrap().clone()
63     }
64 }

```

1.7 アプリケーションの引数管理 options.rs

```
1 use clap::Parser;
2
3 /// 針時計を表示するアプリ
4 #[derive(Parser, Debug)]
5 #[clap(author, version, about, long_about = None)]
6 pub struct Options {
7     /// アプリケーションをデーモン化して実行する
8     #[arg(short, long)]
9     pub daemon: bool,
10    /// info レベルログの出力を行う。(--debug を優先する)
11    #[arg(long)]
12    pub info: bool,
13    /// debug レベルのログの出力を行う。
14    #[arg(long)]
15    pub debug: bool,
16 }
```